

Current

Result

1227 - flash_attention_kernel

Size

(4, 4, 1)x(32, 1, 1)

Cycles

1.14 ms

GPU

0 - NVIDIA GeForce RTX 2070

SM Frequency

1.42 Ghz

Process

[82653] python3.13

Attributes

Summary

Details

Source

Context

Comments

Raw

Session

The report contains imported source files.

GPU Speed Of Light Throughput

GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	1.93	Duration [ms]	1.14
Memory Throughput [%]	15.82	Elapsed Cycles [cycle]	1,615,582
L1/TEX Cache Throughput [%]	36.54	SM Active Cycles [cycle]	714,331.36
L2 Cache Throughput [%]	0.24	SM Frequency [Ghz]	1.42
DRAM Throughput [%]	0.60	DRAM Frequency [Ghz]	6.79

Small Grid

This kernel grid is too small to fill the available resources on this device, resulting in only 0.4 full waves across all SMs. Look at [Launch Statistics](#) for more details.

Key Performance Indicators

Roofline Analysis

The ratio of peak float (FP32) to double (FP64) performance on this device is 32.1. The workload achieved close to 0% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the [Profiling Guide](#) for more details on roofline analysis.

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [cycle]	20,000	# Pass Groups	1
Maximum Buffer Size [Kbyte]	524.29	-	-

Compute Workload Analysis

Pipe Utilization (Elapsed Cycles)

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed ipc Elapsed [inst/cycle]	0.03	SM Busy [%]	0.70
Executed ipc Active [inst/cycle]	0.06	Issue Slots Busy [%]	0.70
Issued ipc Active [inst/cycle]	0.06		

Low Utilization

All compute pipelines are under-utilized. Either this workload is very small or it doesn't issue enough warps per scheduler. Check the [Launch Statistics](#) and [Scheduler Statistics](#) sections for further details.

Est. Local Speedup: 99.57%

Key Performance Indicators

Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	2.59	Mem Busy [%]	15.82
L1/TEX Hit Rate [%]	0	Max Bandwidth [%]	1.93
L2 Hit Rate [%]	35.30	Mem Pipes Busy [%]	1.93
Local Memory Spilling Requests	0	Local Memory Spilling Request Overhead [%]	0

Shared Load Bank Conflicts

The memory access pattern for shared loads might not be optimal and causes on average a 16.8 - way bank conflict across all 538880 shared load requests.This results in 8507392 bank conflicts, which represent 94.04% of the overall 9046272 wavefronts for shared loads. Check the [Source Counters](#) section for uncoalesced shared loads.

Est. Speedup: 34.37%

Key Performance Indicators

Shared Store Bank Conflicts

The memory access pattern for shared stores might not be optimal and causes on average a 14.9 - way bank conflict across all 22816 shared store requests.This results in 317440 bank conflicts, which represent 93.29% of the overall 340256 wavefronts for shared stores. Check the [Source Counters](#) section for uncoalesced shared stores.

Est. Speedup: 34.09%

Key Performance Indicators

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	1.00	No Eligible [%]	93.55
Eligible Warps Per Scheduler [warp]	0.06	One or More Eligible [%]	6.45
Issued Warp Per Scheduler	0.06		

Issue Slot Utilization

Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 15.5 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 8 warps per scheduler, this workload allocates an average of 1.00 active warps per scheduler, but only an average of 0.06 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the [Warp State Statistics](#) and [Source Counters](#) sections can help, too.

Est. Local Speedup: 84.18%

Key Performance Indicators

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	15.52	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	15.52	Avg. Not Predicated Off Threads Per Warp	30.28

Short Scoreboard Stalls

On average, each warp of this workload spends 9.6 cycles being stalled waiting for a scoreboard dependency on a MIO (memory input/output) operation (not to L1TEX). The primary reason for a high number of stalls due to short scoreboards is typically memory operations to shared memory. Other reasons include frequent execution of special math instructions (e.g. MUFU) or dynamic branching (e.g. BRX, JMX). Consult the Memory Workload Analysis section to verify if there are shared memory operations and reduce bank conflicts, if reported. Assigning frequently accessed values to variables can assist the compiler in using low-latency registers instead of direct memory accesses. This stall type represents about 62.0% of the total average of 15.5 cycles between issuing two instructions.

Est. Speedup: 62.04%

Key Performance Indicators

Warp Stall

Check the [Warp Stall Sampling \(All Samples\)](#) table for the top stall locations in your source based on sampling data. The [Profiling Guide](#) provides more details on each stall reason.

Instruction Statistics

Opcode Category Chart

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	1,657,856	Avg. Executed Instructions Per Scheduler [inst]	11,512.89
Issued Instructions [inst]	1,657,984	Avg. Issued Instructions Per Scheduler [inst]	11,513.78

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	16	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	64	Static Shared Memory Per Block [byte/block]	0
Block Size	32	Dynamic Shared Memory Per Block [Kbyte/block]	37.38
Threads [thread]	512	Driver Shared Memory Per Block [byte/block]	0
Waves Per SM	0.44	Shared Memory Configuration Size [Kbyte]	65.54
Uses Green Context	0	Stack Size	1,024
# SMs [SM]	36	# TPCs	18
Enabled TPC IDs	all	-	-

Small Grid

The grid for this launch is configured to execute only 16 blocks, which is less than the GPU's 36 multiprocessors. This can underutilize some multiprocessors. If you do not intend to execute this kernel concurrently with other workloads, consider reducing the block size to have at least one block per multiprocessor or increase the size of the grid to fully utilize the available hardware resources. See the [Hardware Model](#) description for more details on launch configurations.

Est. Speedup: 55.56%

Key Performance Indicators

Occupancy

% Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	3.12	Block Limit Registers [block]	32
Theoretical Active Warps per SM [warp]	1	Block Limit Shared Mem [block]	1
Achieved Occupancy [%]	3.12	Block Limit Warps [block]	32
Achieved Active Warps Per SM [warp]	1.00	Block Limit SM [block]	16

Theoretical Occupancy

The 1.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 8. This kernel's theoretical occupancy (12.5%) is limited by the required amount of shared memory.

Est. Speedup: 84.18%

Key Performance Indicators

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	714,331.36	Average L1 Active Cycles [cycle]	714,331.36
Average L2 Active Cycles [cycle]	27,532.47	Average SMSP Active Cycles [cycle]	178,613.83
Average DRAM Active Cycles [cycle]	46,053	Total SM Elapsed Cycles [cycle]	59,400,084
Total L1 Elapsed Cycles [cycle]	59,400,084	Total L2 Elapsed Cycles [cycle]	49,169,888
Total SMSP Elapsed Cycles [cycle]	237,600,336	Total DRAM Elapsed Cycles [cycle]	61,815,808

SMs Workload Imbalance

One or more SMs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 55.71% above the average, while the minimum instance value is 100.00% below the average.

Est. Speedup: 24.12%

Key Performance Indicators

SMSPs Workload Imbalance

One or more SMSPs have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 88.92% above the average, while the minimum instance value is 100.00% below the average.

Est. Speedup: 9.63%

Key Performance Indicators

L1 Slices Workload Imbalance

One or more L1 Slices have a much lower number of active cycles than the average number of active cycles. Maximum instance value is 55.71% above the average, while the minimum instance value is 100.00% below the average.

Est. Speedup: 24.12%

Key Performance Indicators

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	77,616	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.05	Avg. Divergent Branches [branches]	0

Uncoalesced Shared Accesses

This kernel has uncoalesced shared accesses resulting in a total of 8824832 excessive wavefronts (94% of the total 9386528 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The [CUDA Best Practices Guide](#) has an example on optimizing shared memory accesses.

Est. Speedup: 40.70%

Key Performance Indicators

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
flash_attention_kernel.cu:160 (0x7fc6f261d40 in flash_attention_kernel)	253,952	
flash_attention_kernel.cu:160 (0x7fc6f261d20 in flash_attention_kernel)	253,952	
flash_attention_kernel.cu:160 (0x7fc6f261c00 in flash_attention_kernel)	253,952	
flash_attention_kernel.cu:160 (0x7fc6f261c30 in flash_attention_kernel)	253,952	
flash_attention_kernel.cu:160 (0x7fc6f261c30 in flash_attention_kernel)	253,952	

Follow the rules outputs to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel. You could also disable individual sections to focus on selected performance aspects and make profiling faster.